

YOCO Align 与 Fast : 优化思路、开发过程与验证结果

日期 : 2026-09-05。范围 : vLLM 推理、llm-train 训练与评估、NVIDIA B200。本文汇总 Align 开发与后续训练反向、Fast 专项优化的已完成实验。报告生成脚本本身不运行 GPU 压测。

1. 当前成果

我们已将 YOCO `--align` 从推理侧的 batch invariance 扩展到训练与推理共用前向 kernel，并统一了 log-prob 与交叉熵 (CE) 的概率归约。在已验证的真实 L3、B200、BF16、单卡配置中，两端 logits、完整词表 log-prob，以及 token CE 对负目标 log-prob 均达到逐字节一致。

性能结果如下，比较对象和统计口径各不相同：

层次	已测结果	适用口径
概率算子优化	单行 FP32 完整 log-prob 从旧 vLLM 的 132.61 μ s 降至 18.51 μ s，约 7.16x 加速	CUDA Graph 算子测量；训练 CE 前向加反向额外耗时约 0-1.1%
推理系统	Align 成功请求输出吞吐 582.02 tok/s，历史 Fast 为 1030.41 tok/s，下降 43.5%	同一固定 trace 和历史参数；不同物理 GPU；Align 有 5 条超时并触及并发上限
训练反向优化	512 / 2048 tokens 的整步耗时分别降至 465.3 / 600.1 ms ；相对默认吞吐损失为 7.6% / 16.7% ，比旧 Align 整步快 6.39x / 5.55x	同一物理 B200、BF16、native autograd + SGD、合成输入；最新结果见第 6.4 节
Fast 新增优化	同卡短测 Prefill 吞吐提高 5.3%-8.9% ；长 trace 输出吞吐 1029.27 $\rightarrow$ 1031.97 tok/s (+0.26%)	独立 Fast 分支，BF16 / B200 / TP1；短测 A/B/B/A，长测为同卡基线与候选补跑；见第 6.5 节

MoE 反向已从逐专家循环改为 grouped GEMM 与融合激活梯度，反向相对旧 Align 加速约 **9.08x / 7.09x**。前向函数保持相同；真实 L3 开启梯度的整段前向对逐 token decode 的 **27** 项字节检查、**35** 项测试和 NNScaler 小模型训练复验通过。初版约 85% 的训练吞吐损失保留在第 6.3 节作为历史对照。

Align 的数值验收目标按项目要求限定为前向一致；不同 microbatch 划分的梯度累加、SGD 更新不要求 bitwise。梯度正确性、有限性和训练可用性仍需验证。

2. 目标与比较边界

2.1 我们希望固定什么

对于相同 checkpoint、相同逻辑 token 前缀，改变请求合批方式、目标请求在 batch 中的位置、prefill 分块或执行历史后，应得到相同的前向结果。训练 teacher forcing 与推理 cached decode 也应使用相同算术，在对应预测位置给出相同 logits 和 log-prob。

这里有三个独立问题：

问题	比较对象	本项目状态
Batch invariance	同一实现中的独立请求、合批、分块、KV-cache 执行	已在记录的算子与模型用例中通过
训练／推理前向对齐	llm-train 与 vLLM 对应预测位置的实际输出	最终真实 L3 对照通过
梯度／更新 invariance	一次整批 backward 与多次 microbatch backward 后的梯度和参数	有明确反例，不属于当前 bitwise 目标

严格比较要求 **shape**、**dtype** 一致、全部数值有限、连续 **tensor** 的 **uint8** 视图逐字节相等，并记录 SHA-256。单独使用 `torch.equal` 无法区分 +0 与 -0，也不足以确认 dtype 边界一致；降低精度或放宽容差没有被用作通过判据。

vLLM 内部 LM head 输出 BF16，进入 sampler 前转换为 FP32；训练模型出口为 FP32。验证分别对齐 BF16 投影边界与 FP32 采样输入边界，避免比较不同阶段的 tensor。

2.2 当前支持与验证范围

核心配置为 SM100/B200、YOCO diff_v3/RMSClip、BF16、128 experts、Top-8、单 rank；训练侧 CP=EP=1，当前适配的 hidden size 为 1024/3072、head_dim 为 128。模型为 30A3B-180M-L3/0000-28000 的 merged checkpoint 及对应 HF 导出。

当前结论绑定已记录的设备、依赖、源码与输入范围。尚未完成多卡布局、量化训练、训练 MTP、所有长序列形状以及完整长期收敛验证；不能将已通过的测试矩阵扩展为任意配置的全称保证。

3. 优化思路

3.1 固定数值路径，同时保留可验证的调度优化

浮点加法不满足结合律。batch 变化可能触发不同归约树、GEMM 分块、kernel 分支或 attention 布局，即使数学公式相同也可能产生不同舍入。因此我们逐项固定归约顺序、精度和算术边界，再验证只改变工作调度的优化是否仍能保持相同比特。

没有证据要求关闭的并发能力被保留：最终推理实现仍启用异步调度、decode CUDA Graph 和 shared-expert stream overlap。Fast 的实现策略保持独立。

3.2 前向 kernel 只有一个实现来源

训练侧通过 `llm/arch/align.py` 直接复用 vLLM 已验证的前向 kernel，补充布局转换、保存中间值和 autograd 接口。这样可以减少两套“数学等价但舍入不同”的实现持续分叉。

模块	数值一致性措施	实现取舍
Dense / latent / shared / LM head	固定 invariant GEMM 的 K 维累加路径	小 M 可选更小的 M tile，但必须与原实现逐字节相等
RMSNorm / RMSClip	固定归约，消除小 batch 与大 batch 的算法切换	覆盖 stride 输入及 residual-add 边界
Router	IEEE FP32 projection，固定 softmax、Top-8、归一化和同值选择规则	选中项按 expert ID 排序，固定专家输出求和顺序
Routed experts	固定 Triton W13/W2、weighted SwiGLU 与 Top-8 sum	Align 不走随形状改变的 DeepGEMM W2 分支

模块	数值一致性措施	实现取舍
Attention	统一 paged KV + FA2，固定 split 策略	训练将连续 K/V 做无算术拷贝；backward 使用同次前向保存的 output/LSE
RoPE / diff_v3 / shared SwiGLU	共用 CUDA kernel 和编译上下文	RoPE 缓存共用 CUDA 生成，避开 CPU/GPU 三角函数差异
Log-prob / token CE	共用 FP32 online max/exp-sum 归约和最终减法	完整词表并行输出；指定 token 与 CE 保持融合

Align 策略涉及进程级数值设置，因此模式开关比较使用独立进程。训练只接入需要的概率 forward，保留 ATen 的 log-softmax backward，不将推理专用的全部全局算子替换直接搬入训练。

3.3 先定位第一处偏差，再扩大验证范围

验证从 Linear、Norm、Router、MoE、Attention 逐级推进到模型 hidden/logits、实际 sampler/eval/CE API；同时改变 batch、目标位置、填充行、预热顺序、缓存和进程。这样既能定位偏差来源，也能识别只在某次 JIT 或图捕获历史下“碰巧相等”的实现。

性能测量也分层进行：算子 GPU 时间、带 Python 调度的 eager 时间、固定 trace 的服务表现、真实模型训练步。各层的收益与损失单独报告。

4. 开发与定位过程

阶段一：修复推理 batch invariance

最初发现的偏差来自多处随形状或调度变化的分支：

定位结果	修复措施	对应证据
原 weighted RMSClip 在 108 项中失败 6 项；2048-token、64-head 与 17-token 分块最大差 0.125	所有 M 统一固定 128-wide RMSClip 归约	固定算子矩阵复测通过
独立 prefill 用连续 QKV，和已有 decode 混合后改用 paged KV；首个偏差在 layer 0 attention，616 个元素不同、最大差 0.0078125	纯 prefill 与混合执行都使用 paged KV	最终词表 log-prob 曾出现的 0.811386 最大差在对应复测中消除
扩大 CUDA Graph capture bucket 后，编译的 Router softmax/Top-K 暴露差异	改为固定 Triton routing、明确 tie-breaking 与求和顺序	大 bucket 和真实模型矩阵通过

随后对小 M invariant GEMM 做了局部优化：在不改变 K=64 MMA 遍历的条件下，M≤32 使用 M=16 tile。3072→8192 Q projection 的 CUDA Graph 时间在 M=1 时约从 58.7 μs 降到 17.8 μs，M=8 从 55.3 μs 降到 14.6 μs，并与原 M=128 实现逐位核对。该改进约为 3.3–3.8×，cuBLAS 基线在对应测量中仍更快。[E1]

同阶段也记录了尚未解决的成本：M=2048、3072-wide IEEE Router 约 164 μs，之前的 TF32 projection 约 9.7 μs。large-M GEMM、Router，以及 Align 的 attention／编译策略差异仍需优化，概率 kernel 的加速不能覆盖整个推理路径的成本。

阶段一覆盖了 207 项核心算子检查、真实 3072-wide Router 的额外 18 项，以及全新进程／缓存和反向预热顺序。真实模型覆盖提交 batch 1/2/4/100/1024/2048、目标首／中／尾、4K prefill 对 257-token chunk 和 1024 步采样轨迹。提交请求数与图捕获 bucket 不等于每个调度步实际达到的 runtime batch。

阶段二：接入 **llm-train**

在独立训练分支增加 `--align`、`ModelArgs(align=True)`、`eval` 入口和 `chunked CE` 接口，完成共享前向 kernel 的 `autograd` 适配。首版 `backward` 以验证正确性和控制临时显存为优先：Linear 计算输入／权重梯度，Attention 使用原生 FA2 `backward`，Norm 与激活采用对应表达式重算，MoE 逐 expert 计算。

该阶段小模型 `NNScaler` 编译前后的 loss 均为 **6.2097463608**，一次 SGD 后为 **4.9102153778**，36 个参数梯度张量均有限。真实 L3 的 377/377 参数张量获得有限梯度，短序列 `teacher forcing` 与 `cached decode logits` 对照通过。[E2]

这一步建立了可训练的前向共享接口，也留下了后续性能测试发现的逐 expert `backward` 开销。

阶段三：发现 **logits** 之后的概率实现分歧

更严格的真实 API 审计发现，`logits` 已一致，但实际 `log-prob` 与 `token CE` 尚未对齐：

当时的比较	最大绝对差	说明
llm-train 实际 eval log-prob 对 vLLM sampler log-prob	$3.814697265625 \times 10^{-6}$	两端使用不同 log-softmax 实现
llm-train token CE 对 vLLM 负目标 log-prob	$2.451241016388 \times 10^{-6}$	CE 的归约与减法顺序尚未统一
FP32 模型 logits 对 sampler 输入	0	偏差发生在概率后处理阶段

补充的 54 项 `softmax/log-softmax batch` 检查表明，所测各实现自身在分块／合批后仍可逐字节相等，而 PyTorch 与 vLLM 两种实现相互比较存在差异。由此将问题定位为跨实现算术一致性，没有把它简单归结为 `softmax` 自身不具备 `batch invariance`。[E3]

同阶段的小模型审计还发现：重复同一整批的 36/36 梯度与 SGD 更新相等；整批对两个 `microbatch` 有 34/36 张量不同。前向一致无法消除 `backward GEMM`、归约与低精度梯度累加边界带来的舍入差异。项目随后明确将这项梯度 `bitwise` 要求排除在当前目标之外。

阶段四：统一概率实现，并选择更快的公共归约

比较旧 vLLM、PyTorch 与训练融合 CE 后，选用从训练 CE 提取的 **FP32 online max/exp-sum** 归约，公共源码位于 `vllm/model_executor/layers/yoco_probabilities.py`。vLLM 完整／指定 `token log-prob`、训练 `eval／rolling likelihood` 和融合 CE 共用同一算术。

关键处理包括：

- 1. BF16/FP16 输入先统一为 FP32。原型曾直接归约 BF16，Triton 会因 dtype 选择不同布局；保留这次失败记录，最终固定 FP32 归约。
- 2. `tile/warps` 只依赖词表大小，不按 `batch` 切换数值算法。
- 3. 完整词表输出并行化；指定 token 与 CE 直接输出目标值，避免训练额外生成完整 `log-prob` 矩阵。
- 4. 明确负零约定，使 `CE == -target_logprob` 包括符号位在内一致；支持 `top-k/top-p mask` 后开头整个 `tile` 均为负无穷的情况。
- 5. 保留已有 CE `backward`、`z-loss` 和 `recompute` 语义，并删除重复的 `loss/LSE/z-loss` 分配。

这个选择兼顾了推理概率输出与训练融合 CE。直接用完整 PyTorch log-softmax 再 gather 计算 CE 的前期测量，在 M=2048 时约需 2422 μ s，而原融合 CE 约 226 μ s，因此保留融合接口有实际价值。[E4]

阶段五：端到端测量与比较条件修正

推理先做同卡短 trace 诊断，641 请求下 Fast 759.54、Align 516.07 tok/s，下降约 32.1%。这组是短窗口，客户端参数也与历史长测不同，保留为过程证据。[E5]

随后核对历史 Fast/Qwen 数据，按项目选择只重测 Align，并恢复历史 AIPerf 0.12.0、默认 BOS、并发 512、超时 600 秒、同一冻结 trace、同一 tokenizer 与服务参数；保留历史 Fast 和本轮 Align 不同物理 GPU 的限制。最终推理结论使用这组参数对齐的长 trace，不混合早期 32.1% 结果。[E6]

训练侧另做同物理 GPU、同 checkpoint、同输入的独立进程对照，将前向、反向和 SGD 耗时拆开。512-token 默认基线第一次有一次 725.1 ms 尖峰，因此补跑 20 个正式步骤确认：均值 **430.4 $\pm$ 2.0 ms**，中位数 **429.9 ms**；原始尖峰记录保留。[E7]

阶段六：优化训练 **MoE backward** 并复验前向

针对每步最多 5120 次专家循环，将路由一次按专家统计、排序和 padding，使用 DeepGEMM grouped GEMM 计算 dX、dW13 和 dW2，并复用融合 SwiGLU 梯度。dW2 读取前向真实保存的 BF16 activation，权重梯度以 FP32 累加。额外尝试了连续大 block 搬运；2048-token 诊断未测得额外整步收益，没有将其单独计为加速。

完成空专家、倾斜路由、裁剪、非连续输入、不同 dtype 和 FFN 维度等 35 项测试，再复验 NNScaler 小模型训练与真实 L3 的带梯度前向。最后按相同条件在同一物理 B200 上比较默认、旧 Align 和优化后 Align，得到第 6.4 节的最终结果。[E9]

5. 数值验证结果

最终概率统一后的真实 L3 验证包含 5 组 prompt 长度 67/129/513/4096/1024，共 **320** 个预测位置、每位置 **154,880** 个词表值，即 49,561,600 个 log-prob 值。

检查	最终结果
概率统一前后原始 logits 与生成 token	所测轨迹保持一致
vLLM API 返回 log-prob 对 worker 捕获值	逐字节相等，最大差 0
llm-train 完整 eval log-prob 对 vLLM	320 $\times$ 154,880 全部逐字节相等，最大差 0
token CE 对负 sampler 目标 log-prob	全部逐字节相等，最大差 0
eval/rolling chunk 1、17、128	逐字节相等
融合 LM-head/CE chunk 1、8、16	逐字节相等
最终真实 L3 字节比较	75/75 通过
新增概率测试	15 项通过，包括 dtype、mask、ignore、autograd、CE backward/recompute
相关配置/Align/MTP/attention/eval 回归	155 项通过
最新带梯度真实 L3 前向	27/27 字节检查通过，包括与逐 token decode 的对照

检查	最终结果
最新反向与相关回归	35 项测试通过；NNScaler 小模型编译与 SGD 复验通过
当前训练性能用例	各组预热后 377/377 参数张量梯度存在且有限

此前的推理大 batch、4K chunk、1024 步轨迹与 fresh-cache 矩阵提供了各开发阶段的证据；最终概率替换另跑了上述实际 API 和 75 项字节检查。各阶段结果绑定各自源码快照，测试数量没有相加作为一个最终全覆盖套件。

具体到“逐 token decode 对训练整批前向”：概率统一阶段的完整跨端矩阵在 `torch.no_grad()` 下进行；本轮反向优化后已补充 `model.train()` 且开启梯度的真实 L3 整段 teacher forcing。五组输入、320 个预测位置的 logits、log-prob 和 CE 与保存的 vLLM decode 输出逐字节一致；带梯度／无梯度 hidden、三请求合批目标首／中／尾、82 步逐 token KV-cache 对照也通过，共 27 项检查。NNScaler 编译训练复验使用小模型。真实 L3 完整 NNScaler 生产图、全部序列长度及 batch 组合尚未全面审计，因此结论仍限于已测场景。

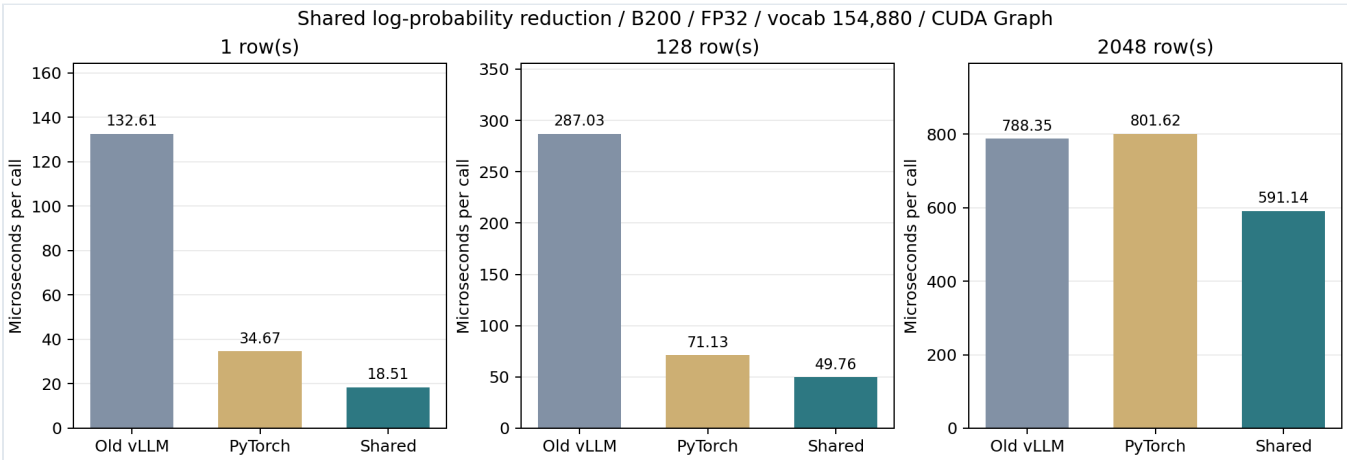
尚未运行完整 `lm_eval` 任务 harness，现有验证覆盖其实际使用的 log-softmax API 与 rolling likelihood 工具。跨运行时采样轨迹比较使用相同逻辑前缀，不承诺不同 sampler 的 RNG 实现完全相同。不同 microbatch 的梯度／更新 bitwise 反例仍保留。

6. 性能结果

6.1 公共概率 kernel

词表大小 154,880，FP32 输入，单位 μ s/调用。此表是 CUDA Graph 下的实际 sampler 完整 log-prob 时间。
[E4]

行数	旧 vLLM	PyTorch	共享实现	相对旧 vLLM 加速
1	132.61	34.67	18.51	7.16×
128	287.03	71.13	49.76	5.77×
2048	788.35	801.62	591.14	1.33×



BF16 输入连同 FP32 转换的共享 sampler 时间为 22.72 / 99.68 / 1393.95 μ s（行数 1/128/2048），在对应 CUDA Graph 档位也优于旧 vLLM 与 PyTorch。共享 CE 的 FP32 forward 额外耗时约 0.6–2.2%，forward+backward 约 0–1.1%。

eager 单行 FP32 调用计入 Python 调度后，共享实现约 51.66 μ s，旧 vLLM 135.18 μ s，PyTorch 36.90 μ s。因此“共享实现最快”的结论限于对应测量条件；单行 eager 仍是 PyTorch 更快。

6.2 推理：按历史 Fast 参数重测 Align

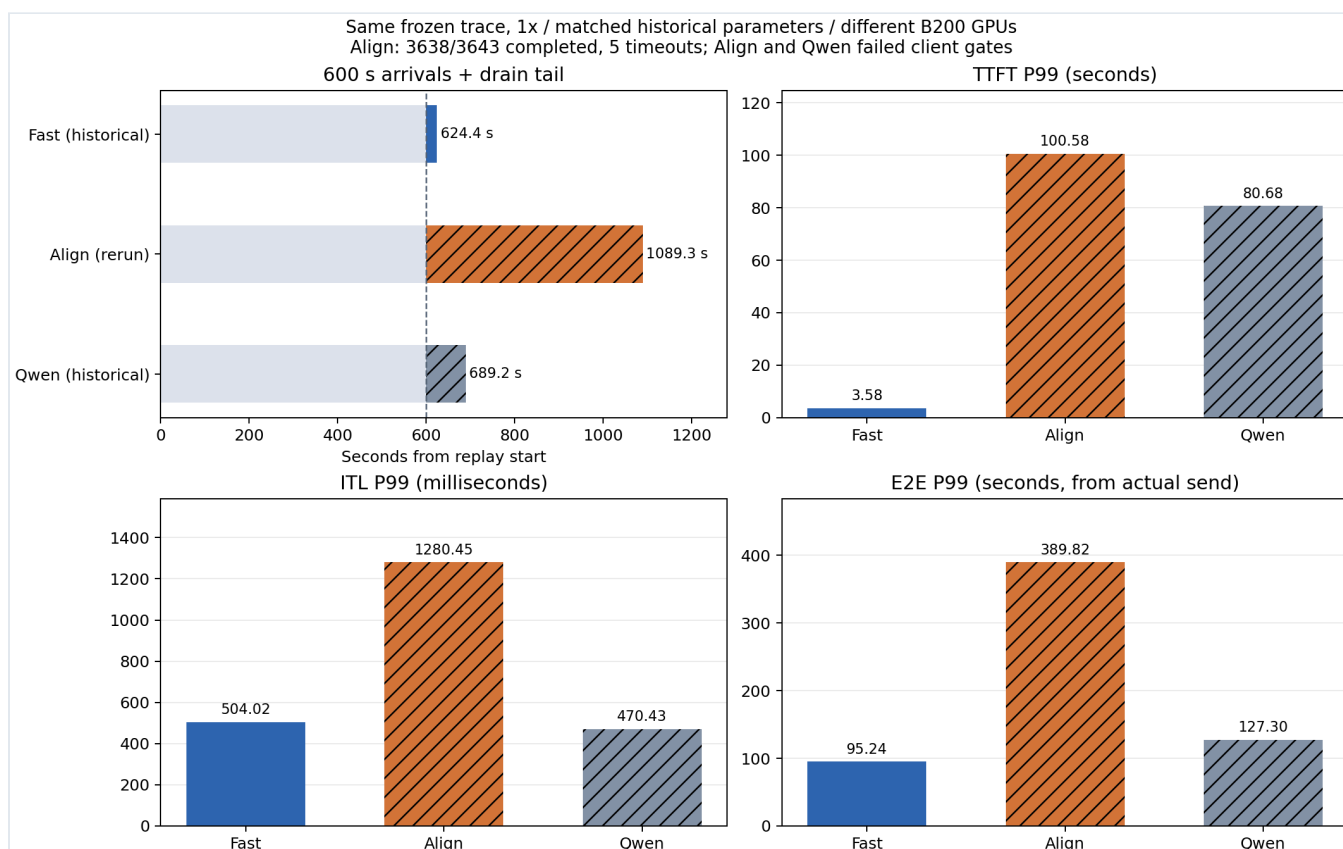
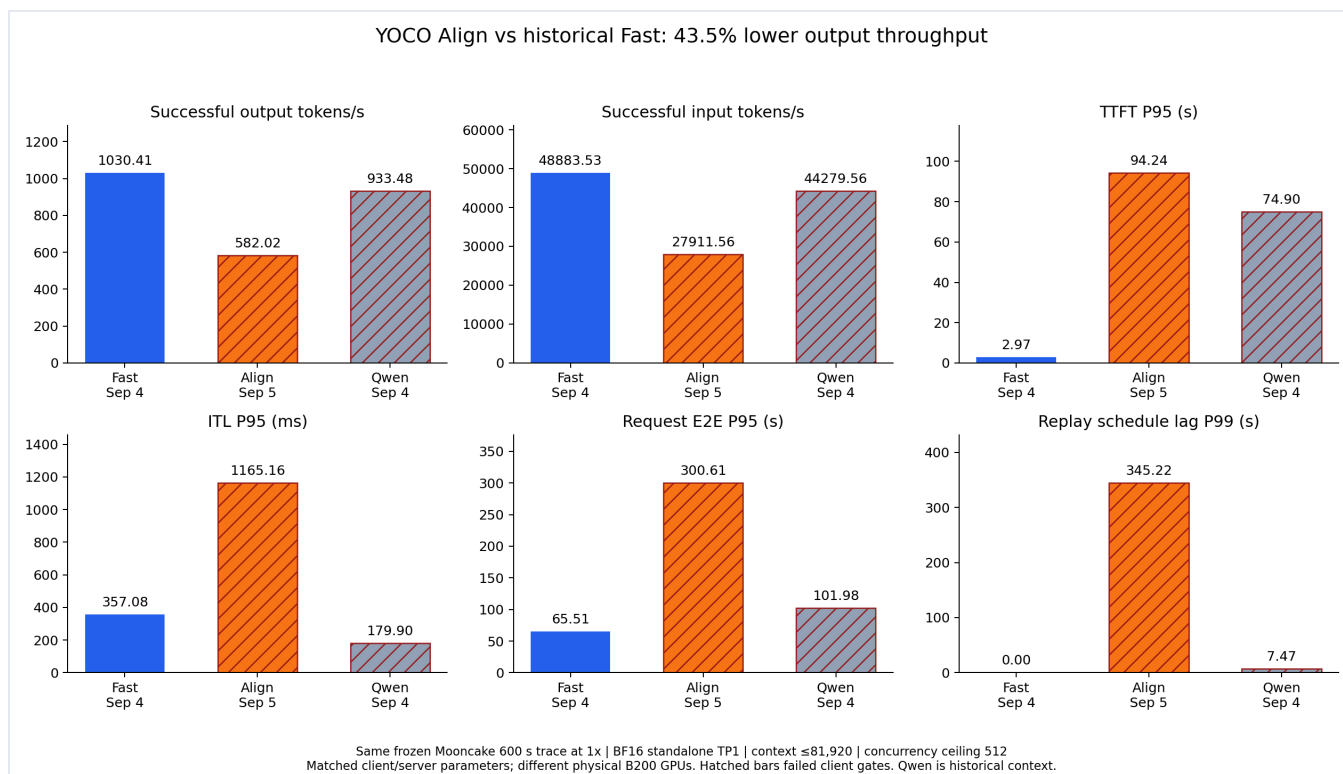
采用 Mooncake FAST’25 toolagent_trace.jsonl，源时间 300-900 秒，1 $\times$ 固定到达计划，3643 请求。原 trace 23608 行，按输入+输出 $\leq$ 81920 保留 23492 行（99.51%），再冻结时间窗口。窗口名义输入约 3052 万 tokens、输出 643375 tokens，计划到达约 6.072 req/s。

Fast 与 Align 使用同一 L3 HF checkpoint、BF16、standalone TP1/DP1，max context 81920、token budget 32768、max sequences 256、显存比例 0.85；prefix cache、chunked prefill、KV-sharing fast prefill 开启，每轮独立 cache salt。请求的 attention/graph 配置保持相同；Align 模式实际选择 FA2、FULL_DECODE_ONLY，并关闭整模型 Inductor，这些模式差异计入测试结果。Qwen 使用其历史模型与运行配置，单独列为参考。

指标	历史 Fast	本轮 Align	历史 Qwen参考
成功/计划请求	3643/3643	3638/3643	3643/3643
错误/超时	0	5	0
成功输出吞吐 tok/s	1030.41	582.02	933.48
成功输入吞吐 tok/s	48883.53	27911.56	44279.56
实际请求吞吐 req/s	5.83	3.34	5.29
运行至排空 s	624.37	1089.25	689.20
TTFT P95 s	2.97	94.24	74.90
ITL P95 ms	357.08	1165.16	179.90
E2E P95 s	65.51	300.61	101.98
调度 lag P99 s	0.003	345.216	7.468
峰值 in-flight / 512	220	512	512
峰值 waiting	20	260	444
排空尾段 s	24.37	489.25	89.21
Prefix cache hit	37.45%	37.30%	35.39%
Preemptions	0	0	54

完整延迟分位数如下；每格依次为 P50 / P95 / P99，E2E 从实际发送开始计时。

延迟指标	历史 Fast	本轮 Align	历史 Qwen参考
TTFT，秒	1.62 / 2.97 / 3.58	71.21 / 94.24 / 100.58	37.86 / 74.90 / 80.68
ITL，毫秒	93.09 / 357.08 / 504.02	407.19 / 1165.16 / 1280.45	80.41 / 179.90 / 470.43
E2E，秒	5.03 / 65.51 / 95.24	89.94 / 300.61 / 389.82	48.41 / 101.98 / 127.30



Align 成功输出吞吐相对 Fast 下降 **43.516%**，统计到排空的时长增加 **74.457%**。由于存在失败请求，这两个比例不表示相同成功工作量的纯速度比，也不能归因于某一个前向 kernel。

本轮 Align 未通过固定到达压力测试：5 条超时、触及 512 上限、回放调度降速。服务端审计通过，metrics 最大间隔 1.024 秒、无 preemption，最终 running/waiting 均归零，但这些不能抵消客户端的失败。普通 E2E 从实际发送计时，不包含客户端等待发送；包含这部分后，估算的“计划到达至完成/超时”P95/P99 约为 500.6/564.9 秒。

历史 Qwen 同样触及并发上限并发生回放降速，仅作为历史系统参考。其相对 Fast 的成功输出吞吐低约 9.4%，模型与系统差异不作为单个 kernel 的因果证据。该组结果没有给出通过指定 SLO 的稳定容量。

用户选择只按历史参数重测 Align，因此保留物理 GPU 与测试时段差异：Fast 为 GPU-f509b284-df92-5434-81e3-ac21d22704d2，Align 为 GPU-14379c29-e601-fc6d-b27c-4fd778ab772a。成功请求的实际 ISL/OSL 逐条匹配历史 Fast；5 条失败请求单独保留。公开 trace 仅含时序、长度与前缀 hash 关系，属于合成回放，不代表真实 agent 任务质量或在线会话能力。[E6]

6.3 初版训练：同一物理 B200 的完整计算步（优化前）

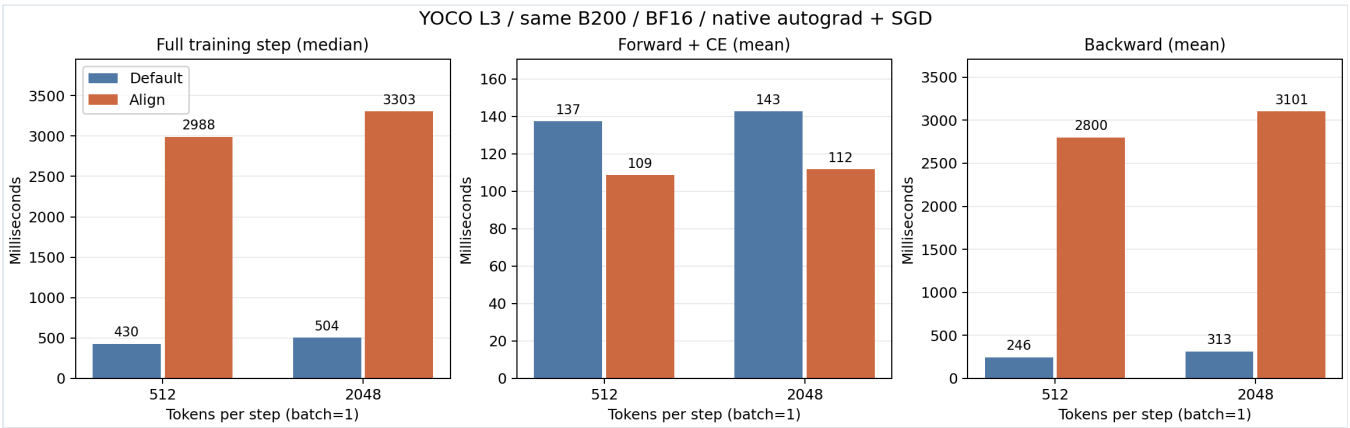
真实 L3 merged checkpoint；BF16 参数（gate FP32）、batch=1、CP=EP=1；相同 seed 的合成 token 和 next-token 标签；CE chunk=128、CE checkpoint 开启；SGD lr=1e-5、foreach=True、无 momentum。关闭 MTP、辅助 MoE loss、fused-FFN recompute 和整模型重计算。

使用 native autograd，保留已有 torch.compile 算子，没有编译完整 NNScaler 图。每组独立进程、重新加载权重；2 步预热、5 步整步测量、另 3 步同步拆项；512 默认确认组取 20 步。整步以 CUDA 边界同步的墙钟时间计时，包含 CPU 调度；加载、编译预热和有限性检查排除在外。

每步 tokens	默认中位数	Align 中位数	耗时倍数	吞吐下降	默认 → Align tok/s
512	429.9 ms	2988.4 ms	6.95×	85.62%	1191.1 → 171.3
2048	504.4 ms	3303.1 ms	6.55×	84.73%	4060.3 → 620.0

吞吐下降按 $1 - \text{默认步耗时} / \text{Align 步耗时}$ 计算；表中 tok/s 用 tokens 除以中位数耗时。

每步 tokens	前向+CE 默认 → Align	反向 默认 → Align	SGD 默认 → Align
512	137.3 → 108.6 ms	245.7 → 2799.6 ms	47.8 → 48.4 ms
2048	142.6 → 111.8 ms	312.6 → 3101.5 ms	47.8 → 48.1 ms



分项使用另 3 步的均值，反向包含 CE checkpoint 重算，因此不与整步中位数强行相加。默认/Align 峰值 allocated 显存分别为：512 tokens 时 133.05/131.27 GiB，2048 tokens 时 139.92/140.44 GiB。

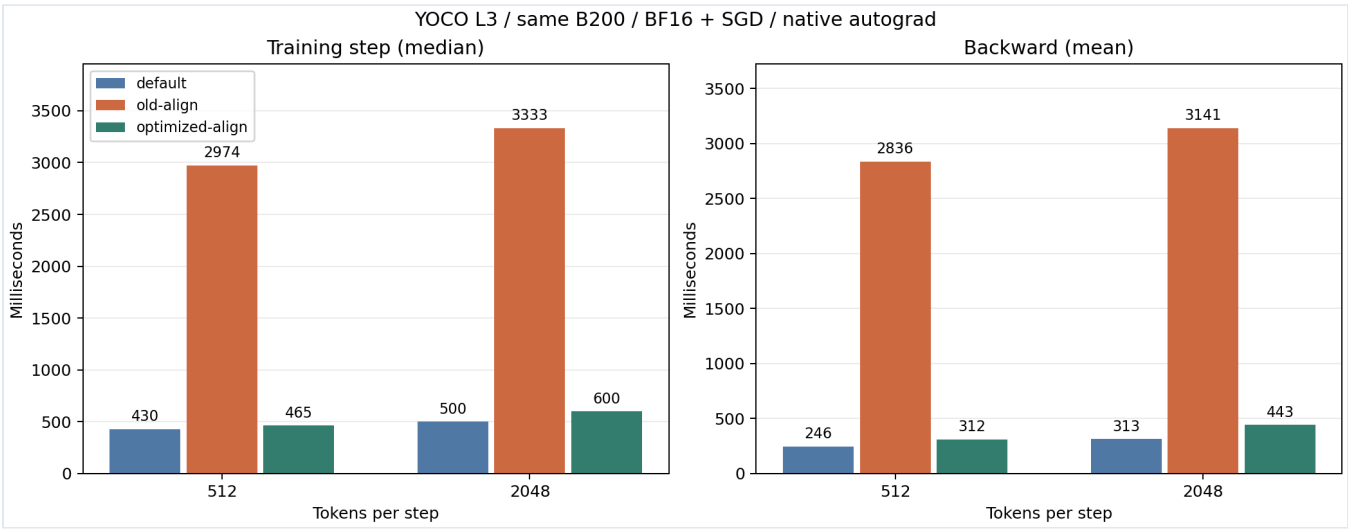
默认 MoE 采用 grouped GEMM 和融合反向；Align 首版则逐 expert 执行 torch.where、多个矩阵乘、激活的 autograd 重算及结果累加。L3 共有 40 次 block 执行，对应每步最多 5120 次专家循环。分项确认反向是主要成本；尚未单独用 kernel profiler 量化 MoE 占全部反向的比例。

这组历史测试量化了初版单卡计算实现的差距。原 checkpoint 训练采用 MXFP8、8192-token 序列、batch=2、MTP=1；本次没有测量该生产配置、多卡通信、AdamW 状态、FP32 master weights 或数据管线。合成输入会影响 MoE 路由分布，节点上其他 GPU 的作业也可能造成 CPU 竞争，结果应保留这些边界。[E7]

6.4 训练反向优化后的结果（最新）

已将逐专家 Python 循环替换为按专家整理路由、DeepGEMM grouped GEMM 与融合 SwiGLU 梯度。dW2 读取前向实际保存的 BF16 activation，权重梯度以 FP32 累加。前向函数及其他 Align 算术的 AST 与改动前保持相同。

每步 tokens	默认	旧 Align	优化后 Align	整步加速	相对默认吞吐损失
512	430.0 ms	2973.6 ms	465.3 ms	6.39×	85.54% → 7.59%
2048	500.1 ms	3333.4 ms	600.1 ms	5.55×	85.00% → 16.67%



这是本轮同一物理 B200 的新测量；默认与最终实现再次成对复测，旧 Align 使用本轮较早的同卡数据。BF16、batch=1、相同真实 checkpoint 与合成输入、native autograd + SGD；每组 2 步预热、10 步计时，表中为中位数。生产 MXFP8、多卡 NNScaler、AdamW 与长期收敛仍不在此性能结论内。

最终 35 项测试通过，包含空专家、倾斜路由、跨 128 行边界、32/64/96 FFN、FP32/BF16 权重及梯度 reference。NNScaler 小模型编译与 SGD 复验通过。六组采用的性能用例均确认 377/377 参数梯度存在且有限。

本轮还扩大了前向验证：真实 L3 在 `train()`、开启梯度的整段 teacher forcing 下，5 组输入、320 个预测位置的完整 logits/log-prob 与冻结的 vLLM decode 结果逐字节一致，CE checkpoint 前向与负目标 log-prob 一致。带梯度/no_grad hidden、三请求合批的目标首/中/尾、82 次逐 token KV-cache 对照均通过，共 27 项字节检查。这扩大了概率统一阶段的 no_grad 验证覆盖；真实 L3 完整 NNScaler 生产图和任意 batch/长度的无条件保证仍未建立。

不同 backward 归约允许小幅浮点误差，后续 SGD 轨迹可能略有差别；前向一致性以相同权重和逻辑输入为前提。尚存的一次专家 counts CPU 传输、padding 及其他反向算子仍可继续优化。连续大 block 搬运的单独尝试没有带来额外可测的整步收益，报告未另计该项收益。

详细日志、补丁和机器可读结果见 [E9]。

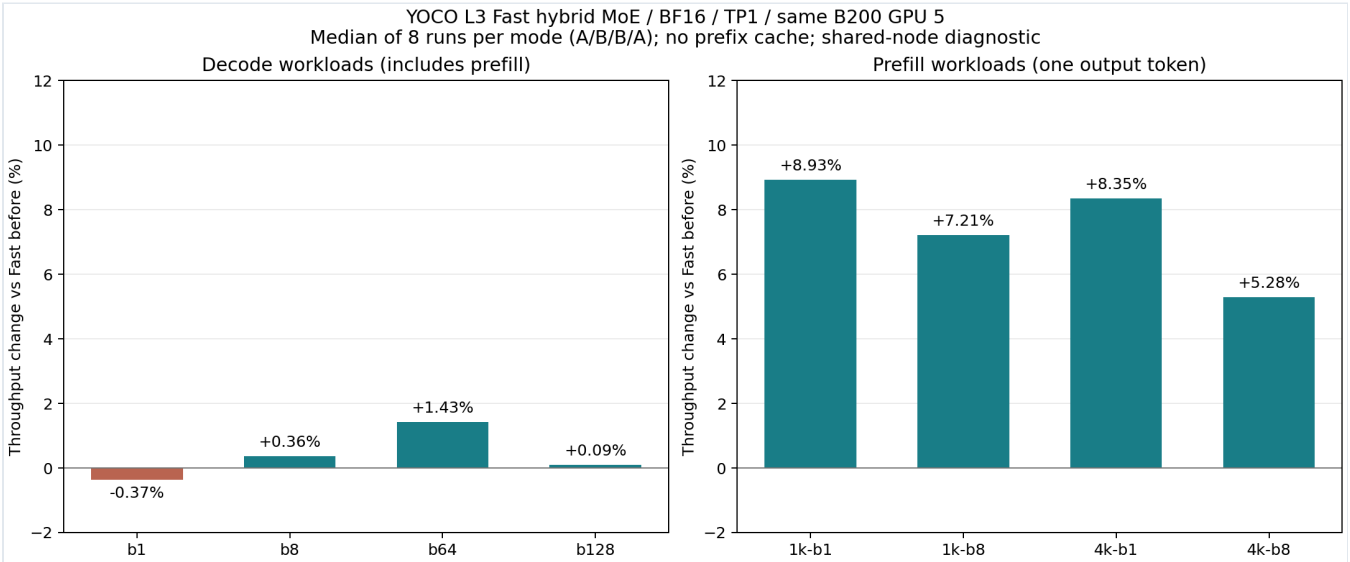
6.5 Fast standalone : 按 token 行数选择 MoE backend

本轮在独立 dev/yoco-fast-optimization-20260905 分支继续优化 Fast。旧 standalone 全程使用 Triton；现在对 L3 BF16、SM100、单 rank、KV-sharing fast-prefill 配置，将 $M \geq 1024$ 的 routed MoE 交给 FlashInfer CUTLASS heuristic，小批次与已配置的 Decode 图保留原 Fast Triton 调参。阈值还会提高到超过 max_num_seqs 和最大 graph bucket，避免扩大配置后误切已有 Decode 图。

两条路径统一处理 W13 的 [up, gate] 布局与 SwiGLU clamp，fallback 使用跨层共享 workspace。早期候选暴露了按 M 取整造成的 workspace 非单调扩容错误，改为精确 flat 容量后，40 项配置/kernel 测试和完整短测通过。原 Align 前向 kernel 与训练分支未改动。[E10]

同物理 GPU 5 的 A/B/B/A，每档每版8次；表中为 batch wall-time 中位数。Decode 用例包含 Prefill，单独 TPOT 的变化约 -0.1% 到 +1.0%，没有测到明确的 Decode 加速。Prefill 四档在两对测量中都改善，汇总吞吐提高5.3%-8.9%。

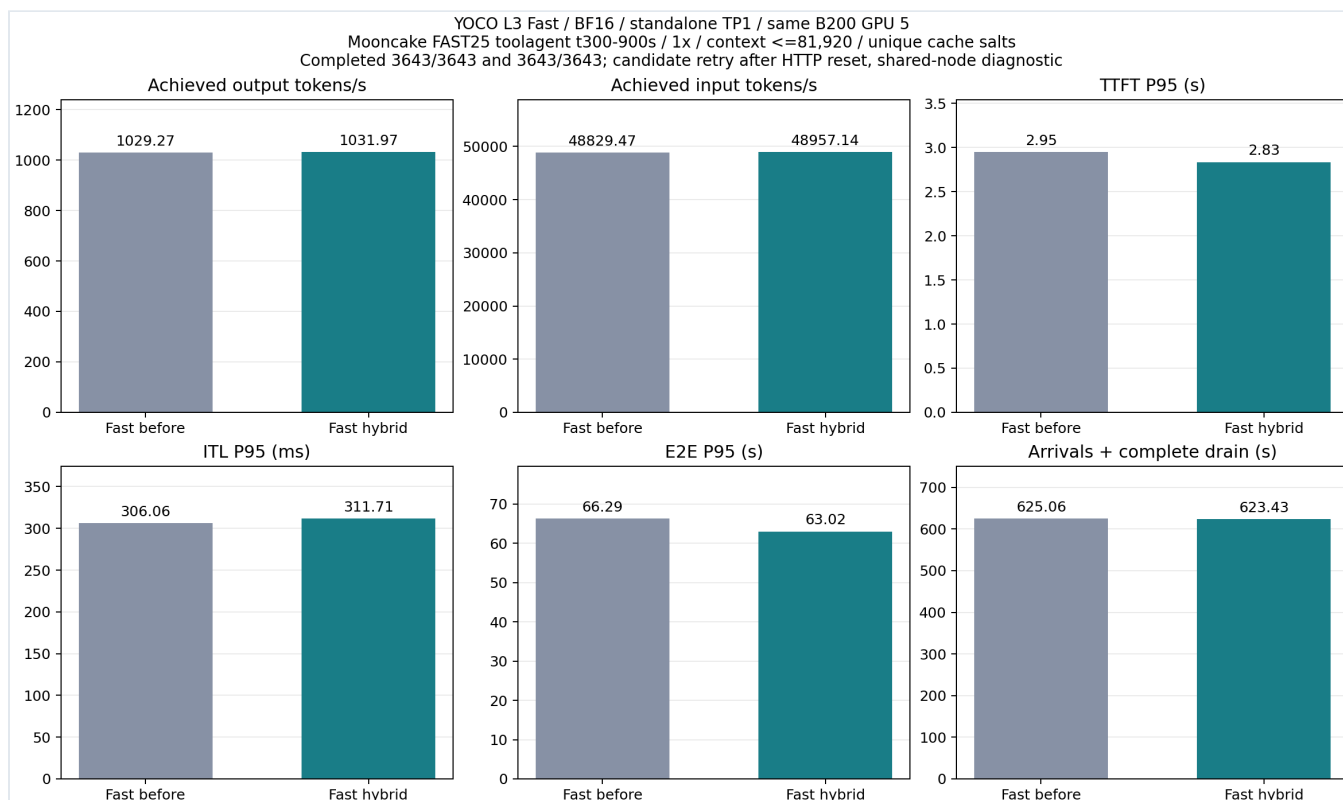
用例	旧 Fast ms	新 Fast ms	吞吐变化
decode-b1	800.704	803.703	-0.37%
decode-b8	1332.282	1327.550	+0.36%
decode-b64	1172.482	1155.949	+1.43%
decode-b128	1546.446	1544.980	+0.09%
prefill-1k-b1	85.851	78.816	+8.93%
prefill-1k-b8	212.850	198.532	+7.21%
prefill-4k-b1	89.787	82.864	+8.35%
prefill-4k-b8	564.737	536.403	+5.28%

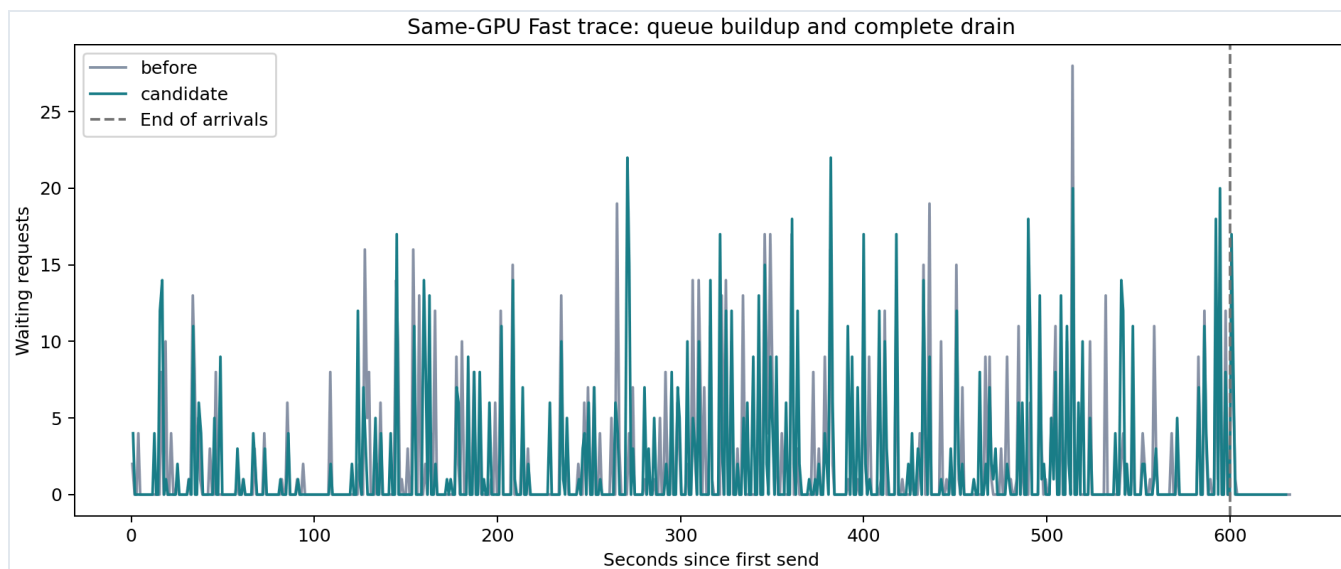


再用同一冻结 Mooncake 600秒到达、3643请求、1xtrace进行同卡前后对照。服务均为BF16、TP1、FA4、maxlen81920、budget32768、maxseq256、mem .85、prefix cache、chunked prefill与KV-sharing fast-prefill、图捕获到256。AIPerf0.12.0、32发送workers、1个record processor、并发安全上限512、超时600秒；各case独立cache_salt。

指标	旧 Fast	新 Fast
完成请求	3643/3643	3643/3643
输出 tok/s	1029.275	1031.966
输入 tok/s	48829.470	48957.140
回放至排空 s	625.060	623.430
TTFT P95 s	2.949	2.834
ITL P95 ms	306.057	311.707
E2E P95 s	66.285	63.022
错误数 / 客户端门禁	0 / True	0 / True

输出吞吐变化 **+0.26%**；TTFT P95、ITL P95、E2E P95分别变化 **-3.92%** / **+1.85%** / **-4.92%**，延迟负值表示改善。两端实际输入/输出计数逐请求相同：**True**。完整P50/P95/P99、排队、调度与缓存指标见专项报告及随附JSON。





这是同卡诊断，主比较使用一次基线与候选的完整补跑，节点其他卡有负载，未声明SLO。首次baseline smoke在服务端完成50条后，AIPerf并行导出停在44条；保留失败，将两端record processor统一为1后重试通过。基线因此多经历了一次smoke计算。候选首轮长测3642/3643成功，有1条复用HTTP连接在响应前被重置，服务端无CUDA/模型错误；首轮输出1031.27 tok/s，但未通过完整性门禁，原始失败保留。候选以相同参数、新cache_salt完整补跑，选取第一次通过的补跑，不按速度择优。cache_salt隔离跨case KV复用；候选补跑前多经历一轮长测，JIT预热历史并不完全相同。固定到达率下的吞吐还受最后请求的排空时间限制，不能据此宣称稳定容量或纯kernel加速比。

Fast不要求bitwise：第一对B64/B128生成轨迹分别有680/4096和1343/8192 token不同；扩展复测也观察到旧Fast自身的高batch变化。数学定义、BF16、128专家、Top-8和clamp保留，尚未做完整模型质量评估。Align的严格前向路径继续独立。本轮Fast默认策略可通过 `yoco_fast_standalone_flashinfer_moe=false` 关闭。

第6.2节的Align损失仍以当时的历史Fast为基线，保留原实验结论；没有把这轮Fast结果直接替换进去计算新的Align损失。

7. 下一步优化方案

MoE backward 批量化和 Fast 大 Prefill 优化已完成，见第 6.4、6.5 节。剩余工作根据 profiler 排序；Align 保持前向逐字节验收，Fast 保留 BF16 语义并验证数值与模型质量。

优先级	方向	实施思路	验收方式
P0	剩余反向开销定位	用 profiler 分离 counts CPU 传输、路由 padding、grouped GEMM、Norm/Router 重算和 CE backward 的成本，确定剩余差距的来源	同卡同输入逐项测量；每项改动通过梯度 reference 和前向字节回归后，再测整步性能
P1	其余 backward 融合	优化 Norm/RMSClip/Router/激活重算，减少通用 autograd 调度与临时 tensor；根据 profiler 决定优先顺序	保留 dtype、裁剪与舍入边界的导数语义；梯度允许经验证的浮点误差，不新增 microbatch bitwise 要求
P1	Align 推理 GEMM / Router	优化固定 K 累加下的 launch、tile、并行度和数据搬运；特别检查 large-M IEEE Router	先逐字节验证再测延迟，覆盖新进程/缓存、反向预热、mixed prefill/decode 与 graph buckets

优先级	方向	实施思路	验收方式
P1	Fast Decode	继续分离 routed MoE、attention 与调度开销，检查匹配 token/路由下的 kernel 时间	同卡同输入重复；报告 TPOT、Prefill 与混合负载，不将生成轨迹变化混作纯 kernel 收益
P2	稳定 trace 性能	完成主要 kernel 优化后，按同一冻结 trace 找到不触及并发上限的负载点，再做 clean-cache 重复	明确 SLO，至少 600 秒到达与完整排空；零超时、无调度降速、lag P99≤500 ms、未触顶；同物理 GPU 的成对复测作为后续方案
P2	生产训练代表性	在可用且已授权的配置下测试完整 NNScaler 图、实际 optimizer、代表性序列和真实训练数据；分布式支持单独开发	报告实际全局 batch、microbatch、并行布局、重计算、显存、通信及 tokens/s；增加收敛检查

重新使用默认快速 backward 时，需要核对其接受的中间值、布局、激活裁剪和精度语义是否与 Align forward 一致。前向固定并不意味着任意 backward 都可直接替换；应复用数学与实现边界兼容的部分，再用独立梯度 reference 验证。

最新 2048-token 用例中，优化后反向约 442.9 ms，默认约 312.9 ms；整步分别约 600.1 ms 与 500.1 ms。剩余差距需要进一步归因，尚未承诺生产训练或下一轮优化后的具体速度。

8. 代码与实验资产

项目	位置／状态
vLLM 分支	dev/yoco-align-invariant-20260905，工作区 vllm_yoco_align_invariant
Fast 优化分支	dev/yoco-fast-optimization-20260905，工作区 vllm_yoco_fast_optimization
llm-train 分支	dev/llm-train-align-20260905，工作区 llm-train-align-invariant
训练前向／autograd 适配	llm-train-align-invariant/llm/arch/align.py
Grouped MoE backward 与测试	llm-train-align-invariant/llm/kernel/align_moe_backward.py、 llm/tests/test_align_moe_backward.py
公共概率归约	vllm_yoco_align_invariant/vllm/model_executor/layers/yoco_probabilities.py
训练 CE 接入	llm-train-align-invariant/llm/kernel/cross_entropy.py、 vocab_parallel_cross_entropy.py
训练／评估入口	llm/nnscale_train.py、llm/eval.py、llm/eval_utils.py
提交状态	开发改动保留在独立 worktree，尚未提交或推送
主要验证 Pod	oidc@msr02 / bonete01 / yoco-align-prob-b200-20260901-master-0
主要节点	slc01-cl02-hgx-0228

原 checkpoint 未被改写。本轮 Fast 的8670临时服务也已完成测试并退出，GPU 5已释放；原Pod UID、restart count与8600/8610/8620服务保持。最终训练测试进程已退出，GPU 5 释放；上一轮自有临时 AIPerf 服务为训练测量腾出了该卡，原有 8600/8610/8620 服务保留。早期报告中的“测试后保留 8660 服务”是当时状态，已被后续训练测量更新。

训练命令从 checkout 的 `llm/` 运行。验证使用 `/data/yoco-align-invariant-20260905/.venv/bin/python`，搭配独立 `runtime-train/runtime-vllm` 与 `NNScaler`；Pod 中的 `native_probe_bootstrap.py` 仅适配公开 `NNScaler` 缺失的单 rank helper，不能推广成多卡兼容结论。完整命令、源码 SHA、参数、输入 SHA、逐步时间和日志位于各原始结果目录。

证据索引

本报告随附 `metrics.json`、图表和生成脚本；`source-manifest.json` 记录引用文件及 SHA-256。以下链接指向现有本地证据，原始大体积 tensor 与 trace 记录不重复打包。

- **[E1]** 推理 **batch invariance** 与第一轮算子优化：阶段报告，验证资产。
- **[E2]** 训练适配与首次 **NNScaler/L3** 验证：实现说明，首次验证资产；说明文档包含后续概率统一的更新。
- **[E3]** 概率统一前的严格审计与反例：审计报告，softmax 检查，梯度反例。
- **[E4]** 概率统一及最终数值结果：报告，汇总，CUDA Graph 算子数据，eager 数据。
- **[E5]** 早期短窗口诊断：比较 JSON。
- **[E6]** 历史 **Fast** 参数对齐后的长 **trace**：完整报告，比较与审计数据，冻结 trace manifest。P50/P95/P99 全量指标也随附于本报告 `metrics.json`。
- **[E7]** 同卡训练性能：报告，完整比较数据，可复现 benchmark。
- **[E8]** 公开 **trace** 来源：Mooncake FAST'25 toolagent trace。本轮冻结窗口 SHA-256：
`680e526d49545258c0ca5b635d0004a44cce2ce990d533ac32024cefee1f0170`。
- **[E9]** 最新反向优化与带梯度验证：反向优化报告、三组同卡数据、带梯度前向验证、增量补丁。
- **[E10]** **Fast standalone** 新优化：专项报告、短测结果、同卡长trace、增量补丁。

正文与图表已嵌入本 HTML，可离线阅读或通过浏览器打印。证据链接指向原工作区，迁移单文件后需要另行保留原结果目录。